Machine Learning Systems and Hardware (COMP70114)

Coursework: CUDA Kernel Development Report

Author
Anas Khan

November 27, 2025

1 Introduction: Part 1

This report details our approach towards implementing Batch Normalisation Inference for tensors in NCHW (Batch, Channel, Height, Weight) memory layout using a CUDA kernel.

1.1 Correctness and Performance Metrics

We were able to achieve the **Time Cost** of **0.983 ms** through this implementation which passed all test cases with efficiently across all the benchmarks (see Figure 1).

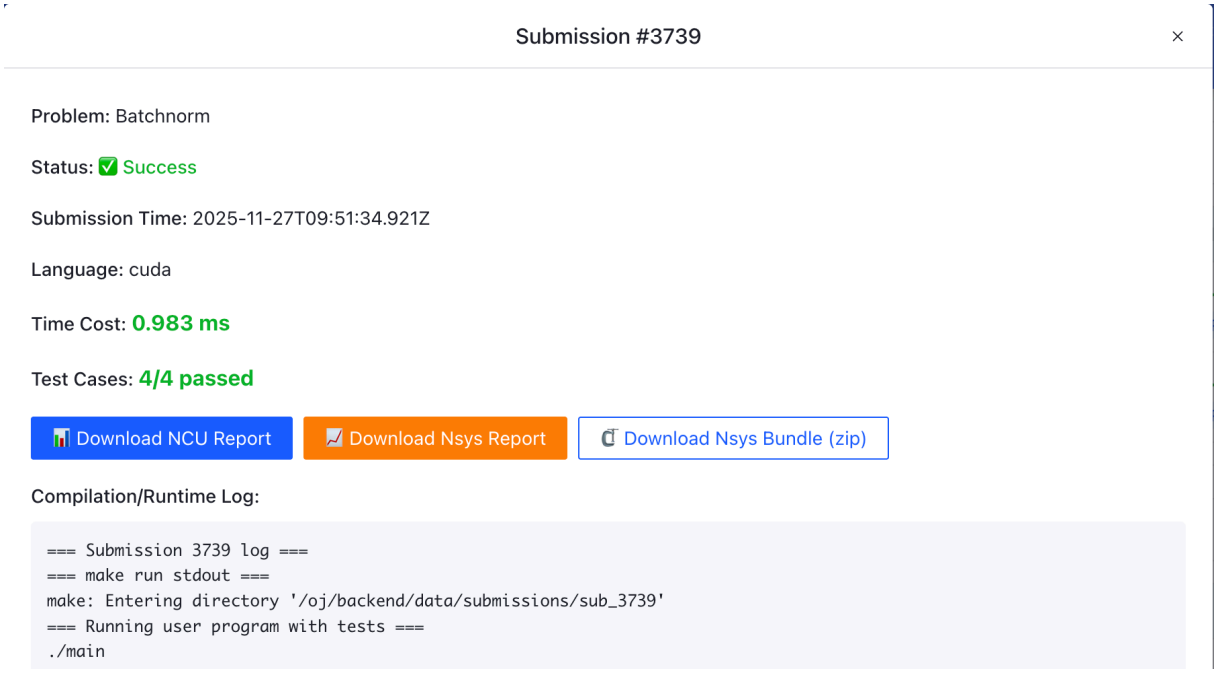


Figure 1: The evaluation of the problem one achieved a time cost of 0.983ms

1.2 Brief Implementation Explanation

- **Architectural Strategy (One-Block-Per-Channel):** The grid dimension was $(C, 1, 1)$. This mapped one CUDA Thread Block distinctively to one Feature Channel. This approach allowed a block to stay alive to process the entire spatial $(H \times W)$ and batch (N) domain for that specific channel.

- **Vectorised Global Memory Access (Memory Coalescing):** The use of the input pointer to `float4*` allowed each thread to load 4 conseuctive floaring point (16 bytes) in a single function.
- **Register-Level Accumulation (Loop Unrolling):** During the accumulation phase, the sums and squared sums are stored in local registers rather than wrting into shared memory. This reduces shared memory bank conflicts.

2 Introduction: Part 2

In the second part the kernel, implements a direct standard convolution algorithm optimized for *Depthwise* layers. In a Depthwise 2D Convolution, the channel interaction is removed; each input channel is convolved with its own unique 2D filter to produce one output channel.

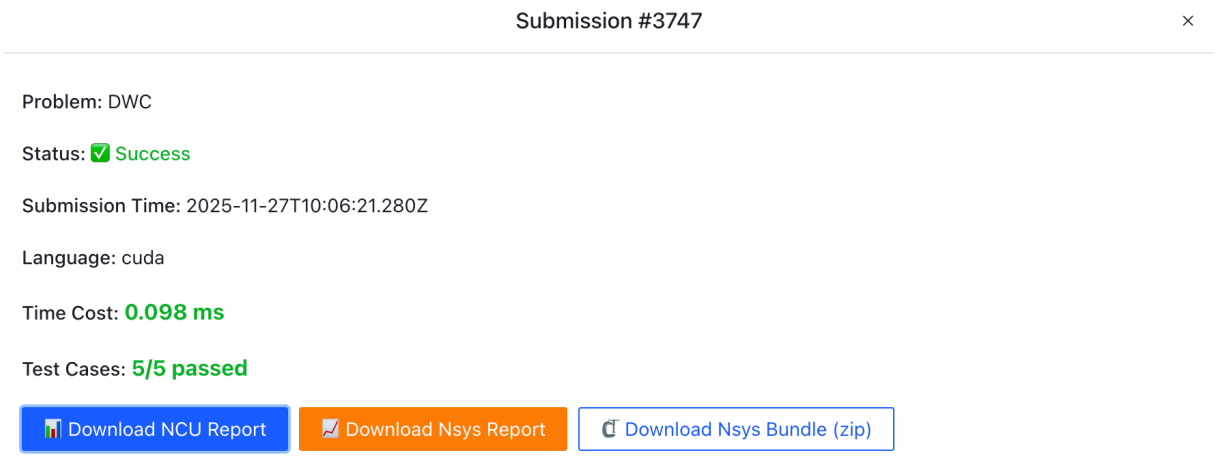


Figure 2: The evaluation of the problem two achieved a time cost of 0.098ms

2.1 Correctness and Performance Metrics

The best submission passed all the test cases efficiently and achieved a promising execution time of **0.098 ms** (see Figure 2).

2.2 Brief Implementation Explanation

- The kernel employs a **One-Thread-Per-Output-Pixel** strategy. This is a common baseline approach where parallelism is achieved by assigning independent threads to calculate every single pixel in the output tensor Y.

  This mapping ensures that every output pixel is computed by exactly one thread, avoiding race conditions (no atomics needed).

- Global Memory Access Pattern: The spatial indexing strategy (x dimension maps to OW) promotes memory coalescing.

  When loading x, threads access $x[\cdots + (ow \cdot stride + kw)]$. If $stride = 1$, these are perfectly contiguous addresses.